Aplicações Frontend

AULA 2 – Fundamentos de HTML5 CANVAS e JavaScript – PARTE 2

Prof. João Paulo Pimentel

joao.pimentel@projecao.br

Roteiro da Aula

- Fundamentos de HTML5 CANVAS e JavaScript
- Arcos e circunferências
- Desenhando Imagens
- Save e Restore
- Animação com requestAnimationFrame
- Orientação a objetos com JavaScript
- Exercícios práticos

Arcos e Circunferências

- arc(x, y, raio, angInicial, angFinal)
  - Ângulos em radianos
  - 2 * Math.PI = 360°

O método `arc` recebe os seguintes parâmetros:

- *x, y:* as coordenadas do ponto central da circunferência;
- *raio* da circunferência em pixels;
- *ângulo inicial* em radianos;
- *ângulo final* em radianos;
- *sentido:* aqui você pode passar `false` (sentido anti-horário) ou `true` (sentido horário). Este parâmetro é opcional; se omitido, o desenho é feito no sentido anti-horário.

Arcos e Circunferências

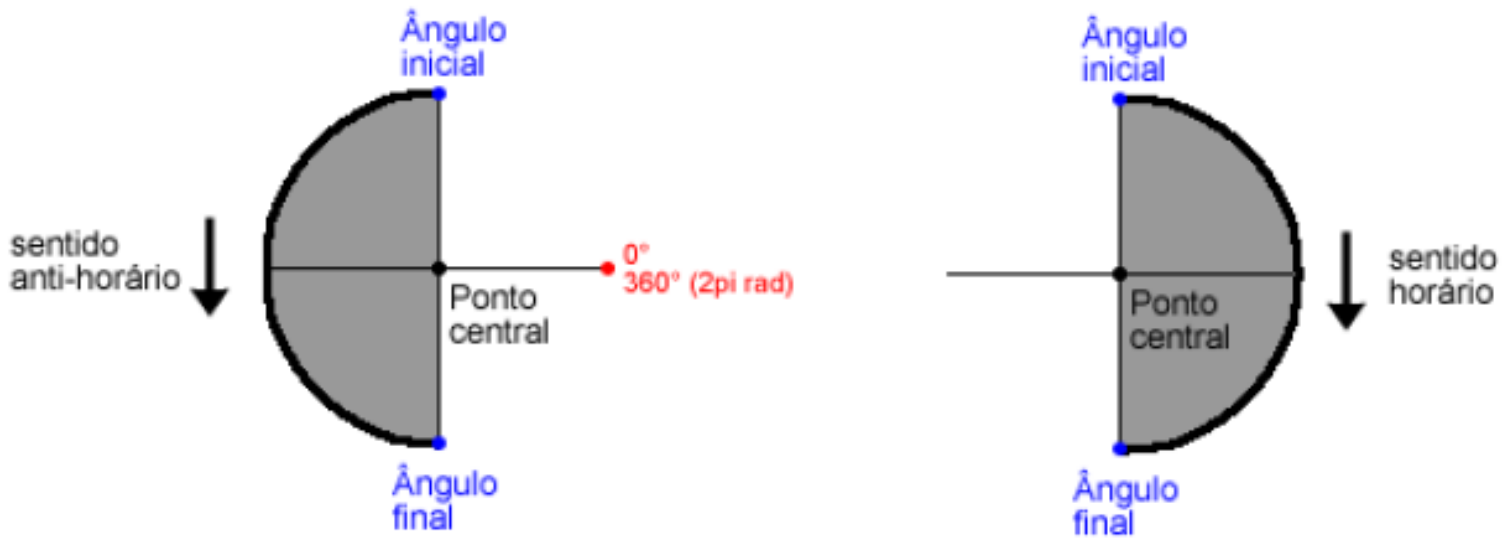

Figura 1.7: Localização do ponto central, ângulos inicial e final e o ponto zero

Arcos e Circunferências

```
// Exemplo teórico
context.arc(
    50,                 // X (centro)
    50,                 // Y (centro)
    40,                 // Raio
    90*Math.PI/180,     // Início: 90 graus
    270*Math.PI/180,    // Término: 270 graus
    false               // Sentido anti-horário
);
```

Arcos e Circunferências

- Vamos experimentar na prática!
- Primeiro, vamos inicializar as variáveis e configurar o contexto com a aparência desejada:

Figura 1.8: Dois arcos desenhados juntos no mesmo path: um é a continuação do outro!

Arcos e Circunferências

```
<!-- arquivo: arcos.html -->
<canvas id="meu_canvas" width="300" height="300"></canvas>
<script>
   // Canvas e contexto
   var canvas = document.getElementById('meu_canvas');
   var context = canvas.getContext('2d');

   // Cores e espessura da linha
   context.fillStyle = 'gray';
```

Arcos e Circunferências

```
   context.strokeStyle = 'black';
   context.lineWidth = 2;

   // continua...
</script>
```

Para fazer um arco, devemos iniciar um path, chamar o método `arc` e depois fazer `fill` e `stroke` (ou somente um destes dois, se quisermos somente preenchimento ou somente contorno):

```
// Primeiro arco:
// Novo path
context.beginPath();
// Desenha
context.arc(50, 50, 40, 90*Math.PI/180, 270*Math.PI/180,
   false);
// Preenchimento
context.fill();
// Contorno
context.stroke();
```

Arcos e Circunferências

```
// continua...
```

Se quisermos fazer outro arco, temos que iniciar mais um path; do contrário, o desenho formado ficará estranho (figura 1.8). Repare que a posição x foi deslocada para desenharmos à direita do anterior:

```
// Segundo arco
context.beginPath();
context.arc(150, 50, 40, 90*Math.PI/180, 270*Math.PI/180,
   true);
context.fill();
context.stroke();
```

Arcos e Circunferências

```
// Circunferência completa
context.beginPath();
context.arc(250, 50, 40, 0, 2*Math.PI);
context.fill();
context.stroke();
```

Pronto! Temos os seguintes desenhos na página:

Figura 1.9: Arco em sentido anti-horário, arco em sentido horário e circunferência completa, com preenchimento e contorno

Desenhando Imagens

- new Image()
  - img.onload
  - drawImage()
  - Clipping (sprites)

```
// Exemplo teórico

// Carregando uma imagem programaticamente
var imagem = new Image();
imagem.src = 'minha-imagem.png';  // gif, jpg...

// Obtendo de uma imagem na página
var imagem = document.getElementById('id_da_tag_img');
```

Desenhando Imagens

- new Image()
  - img.onload
  - drawImage()
  - Clipping (sprites)

Convém aguardar a imagem ser carregada antes de desenhá-la. O objeto `Image` possui o evento `onload`, que será disparado automaticamente pelo browser quando o carregamento estiver completo:

```
// Exemplo teórico
imagem.onload = function() {
   // Aqui trabalhamos com a imagem
}
```

Desenhando Imagens

Estando carregada, a imagem pode ser desenhada através do método `drawImage` do `context`. Este método pode ser chamado de duas formas:

- `drawImage(imagem, x, y, largura, altura)`: desenha a imagem inteira, na posição e tamanho especificados;
- `drawImage(imagem, xOrigem, yOrigem, larguraOrigem, alturaOrigem, xDestino, yDestino, larguraDestino, alturaDestino)`: desenha parte da imagem.

Desenhando Imagens

```
<!-- arquivo: imagens-1.html -->
<canvas id="meu_canvas" width="500" height="100"></canvas>
<script>
   // Canvas e contexto
   var canvas = document.getElementById('meu_canvas');
   var context = canvas.getContext('2d');

   // Carregar a imagem
   var imagem = new Image();
   imagem.src = 'img/ovni.png';
   imagem.onload = function() {
      // Aqui usaremos drawImage
   }
</script>
```

Desenhando Imagens

No evento `onload`, fazemos um loop para desenhar cinco OVNIs, um ao lado do outro. A variável `x` indica a posição de cada desenho:

```
imagem.onload = function() {
   // Começar na posição 20
   var x = 20;

   // Desenhar as imagens
   for (var i = 1; i <= 5; i++) {
      context.drawImage(imagem, x, 20, 64, 32);
      x += 70;
   }
}
```

Desenhando Imagens

Se quiséssemos desenhar as imagens ampliadas ou reduzidas, bastaria modificar a largura e a altura:

```
// Reduzindo para metade do tamanho
context.drawImage(imagem, x, 20, 32, 16);

// Ampliando para o dobro do tamanho
context.drawImage(imagem, x, 20, 128, 64);
```

Vamos experimentar a segunda forma, a que recebe oito valores! Na pasta `img`, temos o arquivo `explosao.png`, que também será usado no jogo. Esta imagem contém uma sequência de animação (*spritesheet*), da qual desenhamos uma parte por vez:

Desenhando Imagens

A técnica de *clipping* consite em selecionar uma área da imagem original para ser desenhada:

Figura 1.12: Selecionando uma área da imagem para recorte (*clipping*)

Os quatro primeiros valores passados ao `drawImage` indicam o retângulo da área enquadrada; os outros quatro representam a posição e o tamanho do desenho no Canvas:

Desenhando Imagens

```
<!-- arquivo: imagens-2.html -->
<canvas id="meu_canvas" width="300" height="300"></canvas>
<script>
   // Canvas e contexto
   var canvas = document.getElementById('meu_canvas');
   var context = canvas.getContext('2d');

   // Carregar a imagem
   var imagem = new Image();
   imagem.src = 'img/explosao.png';
   imagem.onload = function() {
      context.drawImage(
         imagem,
         80, 10, 60, 65,  // Área de recorte (clipping)
         20, 20, 60, 65   // Desenho no Canvas
      );
   }
</script>
```

Save() e Restore()

- save() → salva estado atual
  - restore() → retorna estado anterior
  - Evita conflitos entre desenhos

Como exemplo prático, começamos desenhando um quadrado verde. Em seguida, subimos na pilha com `save` e fazemos um quadrado roxo. Depois que retornamos com `restore`, nossa cor atual passa a ser o verde novamente:

Save() e Restore()

```
<!-- arquivo: save-e-restore.html -->
<canvas id="meu_canvas" width="200" height="200"></canvas>
<script>
   // Canvas e contexto
   var canvas = document.getElementById('meu_canvas');
   var context = canvas.getContext('2d');

   // Um pequeno quadrado verde
   context.fillStyle = 'green';
   context.fillRect(50, 50, 25, 25);

   // Salvamos a configuração e subimos na pilha
   context.save();

   // Agora um quadrado roxo
   context.fillStyle = 'purple';
   context.fillRect(100, 50, 25, 25);

   // Voltamos para o nível anterior na pilha
   context.restore();

   // Voltou para o verde!
```

Save() e Restore()

```
   context.fillRect(150, 50, 25, 25);
</script>
```

Figura 1.16: Salvamos a cor verde na pilha (*save*), mudamos para roxo e voltamos para o verde (*restore*)

Animação com requestAnimationFrame

- Substitui setInterval
  - Mais suave e eficiente
  - Executa no momento ideal do navegador
  - Para demonstrar na prática, vamos fazer uma bolinha se deslocando pela tela.
  - Obtemos as referências do Canvas e do contexto gráfico, definimos a posição inicial da bola e seu raio e, em seguida, mandamos uma animação iniciar, chamando a função mexerBola()

Animação com requestAnimationFrame

```
<!-- arquivo: requestanimationframe.html -->
<canvas id="meu_canvas" width="300" height="300"></canvas>
<script>
   // Canvas e contexto
   canvas = document.getElementById('meu_canvas');
   context = canvas.getContext('2d');

   // Dados da bola
   var x = 20;
   var y = 100;
   var raio = 5;

   // Iniciar a animação
   requestAnimationFrame(mexerBola);

   // Função de animação
   function mexerBola() {
      // Aqui uma bolinha se deslocará
   }
</script>
```

Animação com requestAnimationFrame

```
function mexerBola() {
    // Limpar o Canvas
    context.clearRect(0, 0, canvas.width, canvas.height);

    // Desenhar a bola
    context.beginPath();
    context.arc(x, y, raio, 0, Math.PI*2);
    context.fill();

    // Deslocar 20 pixels para a direita
    x += 20;

    // Chamar o próximo ciclo da animação
    requestAnimationFrame(mexerBola);
}
```

Animação com requestAnimationFrame

Faça experiências! Você pode:

- *mexer para a direita:* some um valor a x (veja a figura 1.2);
- *mexer para a esquerda:* subtraia um valor de x;
- *mexer para baixo:* some um valor a y;
- *mexer para cima:* subtraia um valor de y;
- *mexer na diagonal:* altere as posições tanto de x quanto de y. Você pode somar ou subtrair, usar valores diferentes para cada um etc.

Orientação a objetos com JavaScript

- *Classe:* um tipo de objetos determinado, composto por atributos e métodos definidos;
- *Instância:* cada objeto pertencente a uma determinada classe;
- *Atributo:* cada propriedade dos objetos listada em uma classe;
- *Método:* cada tarefa que um objeto de uma classe pode executar.

Orientação a objetos com JavaScript

- Funções Construtoras

Em linguagens O.O., como Java e C#, definimos classes explicitamente (usando a palavra-chave `class`). Em JavaScript não há classes, mas podemos usar *funções construtoras* para criar os objetos seguindo as classes que projetamos.

Uma função construtora é uma função normal do JavaScript, porém usada com o propósito de criar objetos. No exemplo a seguir, a palavra-chave `this` recebe os atributos de um carro. A palavra `cor`, sozinha, se refere ao parâmetro recebido pela função, mas a construção `this.cor` cria um atributo `cor` no objeto:

Orientação a objetos com JavaScript

```
<!-- arquivo: orientacao-objetos.html -->
<body>
   <script>
      function Carro(cor, velocMaxima) {
         this.cor = cor;
         this.velocMaxima = velocMaxima;
         this.velocAtual = 0;
      }

      // continua...
   </script>
</body>
```

Orientação a objetos com JavaScript

```
// Instanciando objetos da classe Carro
var meuCarro = new Carro('vermelho', 250);
var oponente = new Carro('azul', 300);
```

A expressão `new Carro('vermelho', 250)` cria um novo objeto e executa nele a função construtora. Agora a palavra `this` se referirá a esse objeto!

Já os métodos, em JavaScript, são apenas atributos que referenciam funções. Podemos, por exemplo, criar um método `acelerar` que aumenta em 10 unidades a velocidade do carro:

```
function Carro(cor, velocMaxima) {
   // Criando os atributos
   this.cor = cor;
   this.velocMaxima = velocMaxima;
   this.velocAtual = 0;

   // Criando um método
   this.acelerar = function() {
      this.velocAtual += 10;
   }
}

// ...
```

Chamando o método `acelerar` através da variável `meuCarro` criada anteriormente, esse método vai alterar o atributo `velocAtual` dessa instância, e não da instância `oponente`:

```
// Chamando um método somente em um objeto
meuCarro.acelerar();
```

Orientação a objetos com JavaScript

Por último, mande exibir as velocidades dos dois carros criados. O primeiro acelerou, e o outro não!

```
// Verificando as velocidades
document.write(meuCarro.cor + ': ' + meuCarro.velocAtual);
document.write('<br>');
document.write(oponente.cor + ': ' + oponente.velocAtual);
```

A saída desse código é:

```
vermelho: 10
azul: 0
```

Orientação a objetos com JavaScript

- Objetos sem função construtora

O JavaScript oferece a sintaxe de chaves ({ e }) para criar objetos:

```
var meuObjeto = {
   atributo1: valor,
   atributo2: valor,
   metodo1: function() {

   },
   metodo2: function() {

   }
}
```

Orientação a objetos com JavaScript

- Por exemplo, poderíamos criar um carro da seguinte forma:

```
var meuCarro = {
    cor: 'azul',
    velocidade: 0,
    acelerar: function() {
        this.velocidade += 10;
    }
}
```

Orientação a objetos com JavaScript

- O prototype (protótipo)

Declarar métodos como funções anônimas dentro das construtoras tem um preço: maior consumo de memória. Cada vez que executamos o construtor, uma cópia da função anônima é criada na memória e cada instância terá sua cópia não somente do atributo, mas também da função:

```
function Carro(cor, velocMaxima) {
   // ...

   // Criando método com função anônima
   this.acelerar = function() {
      this.velocAtual += 10;
   }
}
```

Orientação a objetos com JavaScript

- O prototype (protótipo)

Pensando nisso, o JavaScript traz o recurso do *prototype* (protótipo), que é um objeto associado a uma função construtora. Colocando os métodos neste objeto, todas as instâncias usarão as mesmas cópias de cada método.

Vamos nos habituar a colocar a função construtora e o *prototype* de uma classe em um arquivo separado, com extensão `.js`. A palavra-chave `prototype` funciona como se fosse um atributo da função construtora:

```
// arquivo: carro.js
```

Orientação a objetos com JavaScript

- O arquivo carro.js

```
// Função construtora
function Carro(cor, velocMaxima) {
   this.cor = cor;
   this.velocMaxima = velocMaxima;
   this.velocAtual = 0;
}

// Prototype com os métodos
Carro.prototype = {
   acelerar: function() {
      this.velocAtual += 10;
   }
}
```

Orientação a objetos com JavaScript

Escrevemos um pouco mais de código, porém até ganhamos um pouco mais de legibilidade e organização: conseguimos "enxugar" a função construtora, e podemos ter o funcionamento completo de um carro em seu arquivo específico — para *isto* servem as classes! Para utilizar essa classe em uma página HTML, basta usar a tag `<script>` com o atributo `src`, referenciando o arquivo `carro.js`:

Orientação a objetos com JavaScript

```
<!-- arquivo prototype.html -->
<!DOCTYPE html>
<html>

<head>
   <title>O prototype</title>
   <script src="carro.js"></script>
</head>

<body>
   <script>
      var meuCarro = new Carro('vermelho', 250);
      meuCarro.acelerar();
      document.write(meuCarro.cor + ': ' + meuCarro.velocAtual);
   </script>
</body>

</html>
```

BOA NOITE !!!!

Até a próxima aula...